



GIT

Advanced



Table of contents

01

Recap & Review

Git Basic

02

Team working

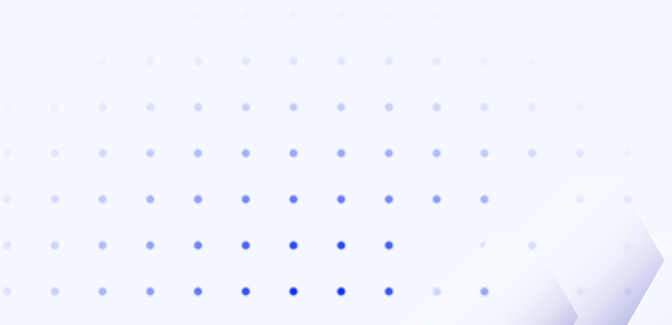
Git Advanced



01

Recap & review

GIT Basic



Nhận code dự án về máy như thế nào?

Bạn được gửi link repository của dự án và muốn tải code vào thư mục onboarding-lab để bắt đầu làm việc.

Lệnh nào phù hợp nhất?

A `git pull <url> onboarding-lab`

B `git clone <url> onboarding-lab`

C `git push <url> onboarding-lab`

Nhận code dự án về máy như thế nào?

Bạn được gửi link repository của dự án và muốn tải code vào thư mục onboarding-lab để bắt đầu làm việc.

Lệnh nào phù hợp nhất?

ĐÁP ÁN B

A `git pull <url> onboarding-lab`

B `git clone <url> onboarding-lab`

C `git push <url> onboarding-lab`

GIẢI THÍCH

`git clone` tạo repository local trong onboarding-lab, gồm lịch sử và bản làm việc. `pull` không tạo bản sao ban đầu.

CẦN NHỚ

`clone`: lấy dự án lần đầu.
`pull`: cập nhật repository đã có.
`push`: gửi commit lên remote.

Kiểm tra thay đổi trước khi staging

Bạn vừa sửa app.py và README.md. Trước khi đưa riêng app.py vào staging area, bạn muốn biết file nào đã đổi và nội dung khác biệt.

Thứ tự lệnh nào hợp lý nhất?

A `git status` → `git diff` → `git add app.py`

B `git add app.py` → `git clone` → `git status`

C `git push` → `git diff` → `git add app.py`

Kiểm tra thay đổi trước khi staging

Bạn vừa sửa app.py và README.md. Trước khi đưa riêng app.py vào staging area, bạn muốn biết file nào đã đổi và nội dung khác biệt.

Thứ tự lệnh nào hợp lý nhất?

ĐÁP ÁN A

A `git status` → `git diff` → `git add app.py`

B `git add app.py` → `git clone` → `git status`

C `git push` → `git diff` → `git add app.py`

GIẢI THÍCH

`git status` báo trạng thái file; `git diff` xem thay đổi chưa staging. `git add app.py` chỉ chọn nội dung của app.py cho commit.

CẦN NHỚ

`git status`: xem trạng thái các file.
`git diff`: xem thay đổi chưa staging.
`git diff --staged`: xem phần đã staging.

Lưu lại một mốc thay đổi có ý nghĩa

Bạn đã staging phần sửa lỗi kiểm tra dữ liệu đăng nhập. Bây giờ cần tạo một snapshot để nhóm dễ theo dõi lịch sử.

Bạn sẽ dùng lệnh nào?

A `git status`

B `git commit -m "Fix login validation"`

C `git diff`

Lưu lại một mốc thay đổi có ý nghĩa

Bạn đã staging phần sửa lỗi kiểm tra dữ liệu đăng nhập. Bây giờ cần tạo một snapshot để nhóm dễ theo dõi lịch sử.

Bạn sẽ dùng lệnh nào?

ĐÁP ÁN B

A `git status`

B `git commit -m "Fix login validation"`

C `git diff`

GIẢI THÍCH

`git commit` lưu snapshot đã staging vào lịch sử local. -m đặt thông điệp; `git status` và `git diff` chỉ dùng để kiểm tra.

CẦN NHỚ

Một commit = một thay đổi có ý nghĩa.
Thông điệp nêu rõ nội dung thay đổi.
Commit lưu local; push chia sẻ remote.

Đồng bộ trước khi đẩy code lên remote

Bạn chuẩn bị đẩy commit lên remote, nhưng đồng đội vừa cập nhật repository. Bạn cũng không nhớ tùy chọn của lệnh pull.

Cách xử lý thực tế nhất là gì?

A `git push` ngay để ghi đè thay đổi

B `git help pull` → `git pull` → kiểm tra → `git push`

C `git clone` lại repository rồi bỏ thư mục cũ

Đồng bộ trước khi đẩy code lên remote

Bạn chuẩn bị đẩy commit lên remote, nhưng đồng đội vừa cập nhật repository. Bạn cũng không nhớ tùy chọn của lệnh pull.

Cách xử lý thực tế nhất là gì?

ĐÁP ÁN B

A `git push` ngay để ghi đè thay đổi

B `git help pull` → `git pull` → kiểm tra → `git push`

C `git clone` lại repository rồi bỏ thư mục cũ

GIẢI THÍCH

`git help pull` tra cú pháp; `git pull` tải và tích hợp thay đổi mới. Xử lý xung đột (nếu có), kiểm tra rồi mới push.

CẦN NHỚ

`pull` = `fetch` + tích hợp vào nhánh.
Merge/rebase theo quy ước nhóm.
Không dùng `force push` tùy tiện.

Thay đổi đi qua ba vùng Git thế nào?

Bạn sửa app.py, chạy git add app.py rồi git commit. Hãy xác định đường đi của thay đổi qua ba vùng làm việc của Git.

Thứ tự nào đúng?

A Working directory → Repository → Staging area

B Working directory → Staging area → Repository

C Staging area → Working directory → Repository

Thay đổi đi qua ba vùng Git thế nào?

Bạn sửa app.py, chạy git add app.py rồi git commit. Hãy xác định đường đi của thay đổi qua ba vùng làm việc của Git.

Thứ tự nào đúng?

ĐÁP ÁN B

A Working directory → Repository → Staging area

B Working directory → Staging area → Repository

C Staging area → Working directory → Repository

GIẢI THÍCH

Sửa file ở working directory; git add chọn phiên bản vào staging area; git commit lưu snapshot đó trong repository local.

CẦN NHỚ

Sửa → add → commit diễn ra local.
Checkout lấy phiên bản đã lưu về working directory.

Commit có lưu mọi thay đổi không?

Bạn sửa app.py và README.md, nhưng chỉ chạy git add app.py. Sau đó, bạn chạy git commit -m "Update app".

Commit này ghi nhận thay đổi nào?

- A Thay đổi của cả app.py và README.md**
- B Chỉ thay đổi đã staging của app.py**
- C Không có thay đổi nào nếu chưa git push**

Commit có lưu mọi thay đổi không?

Bạn sửa app.py và README.md, nhưng chỉ chạy git add app.py. Sau đó, bạn chạy git commit -m "Update app".

Commit này ghi nhận thay đổi nào?

ĐÁP ÁN B

A Thay đổi của cả app.py và README.md

B Chỉ thay đổi đã staging của app.py

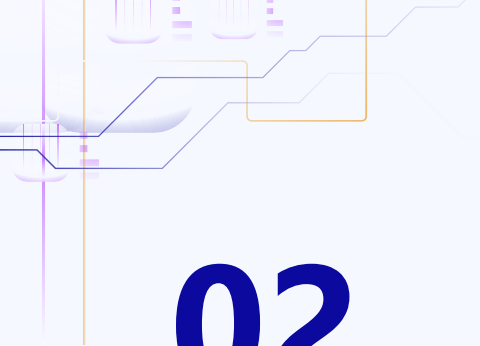
C Không có thay đổi nào nếu chưa git push

GIẢI THÍCH

Chỉ app.py đã được staging nên commit này chỉ ghi nhận thay đổi của app.py. Thay đổi ở README.md vẫn chưa được commit.

CẦN NHỚ

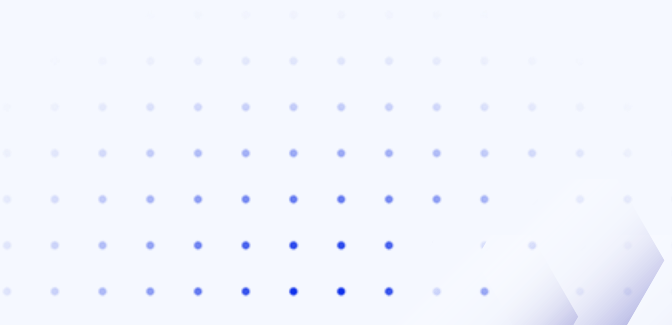
git add chụp nội dung tại lúc chạy.
Sửa tiếp sau add? Cần add lại.
Kiểm tra bằng git diff --staged.



02

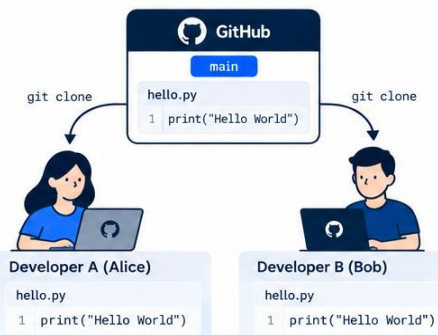
Team working

Git Advanced



Nếu 2 người cùng làm việc trên main?

1 Both developers start from the same main branch



2 Developer A commits and pushes first

Developer A (Alice)

```
hello.py
- print("Hello World")
+ print("Hello from Alice")
```

```
$ git add hello.py
$ git commit -m "Update greeting"
$ git push origin main

Enumerating objects: 3, done.
Writing objects: 100% (1/1), done.
To github.com:user/repo.git
✓ [main] -> main
```

3 A's push succeeds

GitHub

```
main
hello.py
1 print("Hello from Alice")
```

✓ Main now contains:

```
hello.py
1 print("Hello from Alice")
```

4 Developer B has already committed his changes

Developer B (Bob)

```
hello.py
- print("Hello World")
+ print("Hello from Bob")
```

```
$ git add hello.py
$ git commit -m "Update greeting"
[main 1a2b3c] Update greeting
1 file changed, 1 insertion(+),
1 deletion(-)
```

5 Developer B tries to push

```
$ git push origin main
To github.com:user/repo.git
! [rejected]        main -> main
error: failed to push some refs to 'github.com:user/repo.git'
hint: Updates were rejected because the remote contains work that you do not have locally.
hint: Integrate the remote changes (e.g. 'git pull ...') before pushing again.

✗ Push rejected (non-fast-forward)
Main has new changes from Alice that Bob does not have.
```

6 Developer B pulls and gets a merge conflict

```
$ git pull origin main
From github.com:user/repo
* branch          main       -> FETCH_HEAD
Auto-merging hello.py
CONFLICT (content): Merge conflict in hello.py
Automatic merge failed; fix conflicts and then commit the result.
```

hello.py (after git pull)

```
1 <<<<<< HEAD
2 print("Hello from Bob")
3 =====
4 print("Hello from Alice")
5 >>>>>> origin/main
```



Git nên giữ Alice hay Bob?



Git can detect conflicting changes, but it cannot understand developer intent.

Một main chung, bốn rủi ro

Khi main nhận thay đổi trực tiếp,
thiếu review và kiểm tra bắt buộc.

01 push rejected

Remote đã có commit mới;
người push sau cần đồng bộ.

02 merge conflict

Khi sửa trùng đoạn, Git có thể
không tự gộp được lúc tích hợp.

03 code chưa review vào main

Thiếu review bắt buộc: thay đổi
đi thẳng vào code chung.

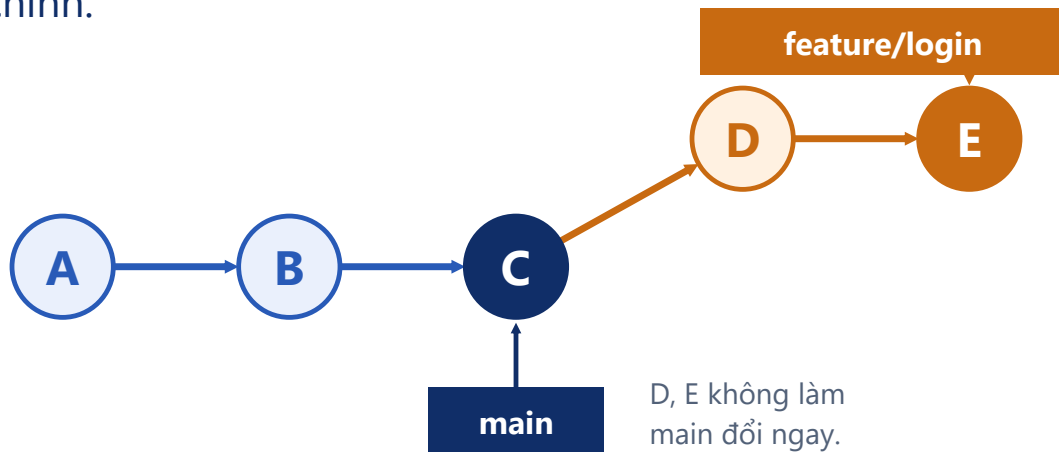
04 main có thể bị broken

Thiếu kiểm tra: lỗi có thể lọt
vào bản code cả nhóm dùng.

Team collaboration requires isolation + review.

Branch là gì?

Branch là một con trỏ (pointer) trỏ đến một snapshot của code tại một thời điểm cụ thể. Nó cho phép bạn phát triển tính năng mới hoặc sửa lỗi mà không tác động đến codebase chính.



Hai branch chung lịch sử đến C.

THỰC HÀNH

git branch

xem danh sách branch

git switch -c feature/login

tạo và chuyển sang branch mới

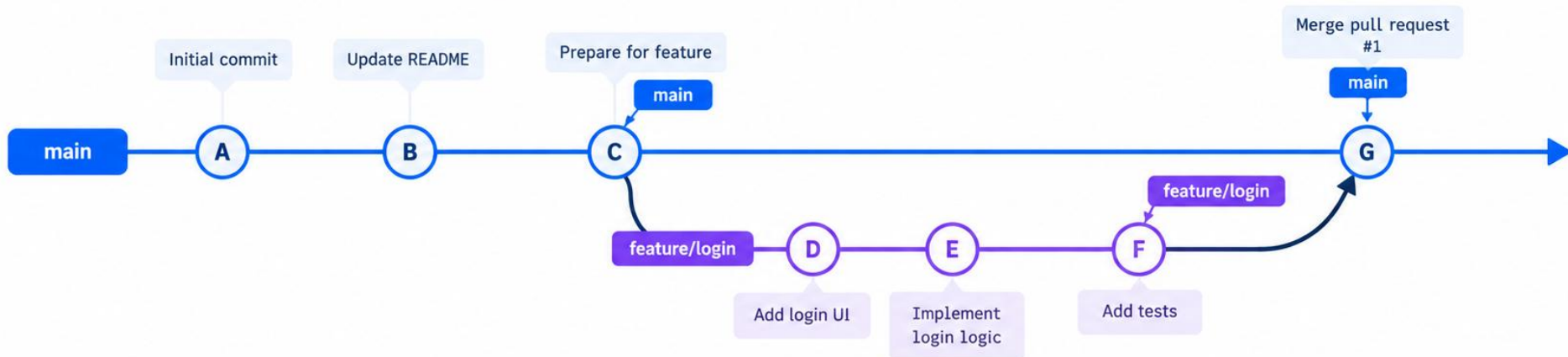
git switch main

quay lại branch main

Tạo branch không copy toàn bộ project.

Branch chỉ là một con trỏ (pointer) tới commit hiện tại.

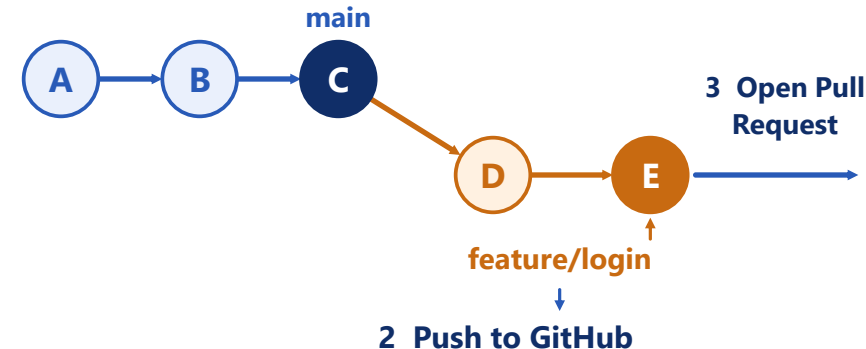
Luồng làm việc tiêu chuẩn với GIT



Pull Request là gì?

Đề nghị review và merge thay đổi từ branch này sang branch khác.

1 Developer · Add login feature



`git push -u origin feature/login`

Add login feature

Source branch

feature/login

✓ 2 commits

Target branch

main

✓ 3 files changed

4 Code Review · Reviewer: Alice

Comments / discussion

✓ Status checks: Checks passed

5 Status: Approved

6 Merge

→ **main**

Why Pull Request?

Review code trước khi merge

Thảo luận thay đổi

Bảo vệ main branch

Pull Request ≠ git pull

git pull: lấy thay đổi từ remote về local

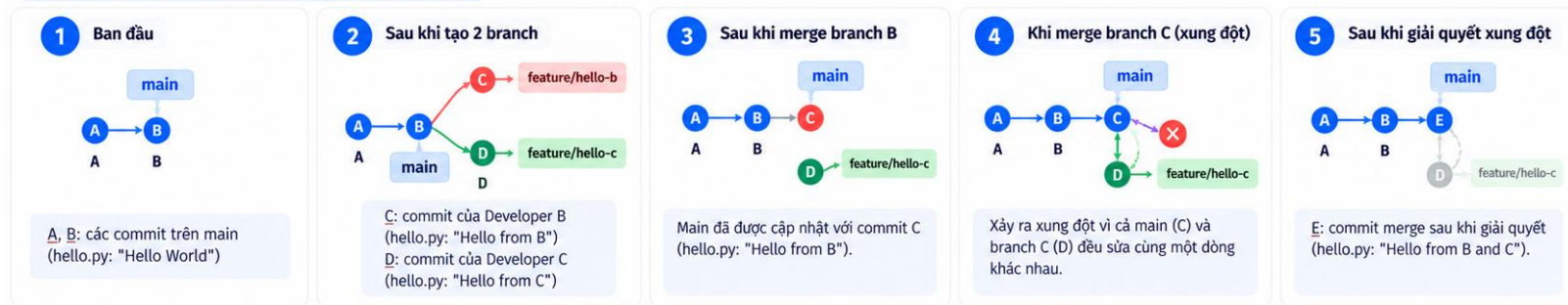
Pull Request: đề nghị review và merge code trên GitHub

Code không đi thẳng vào main — nó được review trước.

Conflict



Sơ đồ branch và commit qua từng bước



Working Convention



1. Đặt tên branch

- Dùng tên rõ ràng theo mục đích
- Ví dụ: `feature/churn-analysis`
`fix/login-bug` `docs/readme-update`
- Thống nhất cách đặt tên trong cả team



2. Commit có ý nghĩa

- Mỗi commit nên thể hiện một thay đổi logic
- Tốt: `feat:` add churn baseline
- Tốt: `fix:` handle missing values
- Tránh message mơ hồ như “update” hoặc “fix stuff”



3. Pull request nhỏ

- Mỗi PR chỉ nên có một chủ đề
- Dễ review và thảo luận hơn
- Merge nhanh hơn, ít conflict hơn
- Ưu tiên thay đổi nhỏ thay vì gộp quá nhiều việc



4. Không đưa secrets hoặc dữ liệu lớn vào Git

- Không commit API key, mật khẩu hoặc file `.env`
- Không lưu raw dataset lớn trong Git
- Dùng `.gitignore` cho file nhạy cảm hoặc file sinh ra tự động
- Lưu dữ liệu ở cloud/storage hoặc công cụ versioning cho data



Mục tiêu: lịch sử rõ ràng hơn, repo an toàn hơn, cộng tác dễ hơn.